ARGOS LiDAR + SLAM Bring-Up Guide

Target: Bench-test the RPLiDAR A1M8, visualize scans in RViz, and run SLAM Toolbox to build a 2D occupancy map.

Starting point: Clean install of Ubuntu on your dev machine and the ARGOS project zip extracted somewhere on disk.

End state: You've verified the LiDAR publishes scans, you can see the scans in RViz, and you've built a saved map of a small area using slam_toolbox.

Time estimate: ~60–90 minutes for the first-time setup, including ROS 2 install.

Section 0 — What we're building

We'll set up a stack that looks like this:

```
┌──────────────┐   /scan    ┌──────────────┐   /map    ┌──────┐
│ rplidar_node │ ──────────▶ │ slam_toolbox │ ─────────▶ │ RViz2 │
└──────────────┘            └──────────────┘            └──────┘
       ▲                           ▲
       │                           │
  /dev/ttyLIDAR            robot_state_publisher
  (CP2102 USB)             (URDF → TF tree)
                                   ▲
                                   │
                         static_transform_publisher
                         (odom → base_footprint, identity)
```

Since ARGOS doesn't have wheel encoders yet, we'll publish an **identity odometry TF** during the bench test and let slam_toolbox's scan matcher estimate motion directly from LiDAR frame-to-frame. This works well for slow, deliberate movement.

Section 1 — Verify Ubuntu version

ROS 2 Humble requires **Ubuntu 22.04 Jammy**. Confirm before proceeding:

```bash
lsb_release -a
```

Expected output includes:

```
Distributor ID: Ubuntu
Release:        22.04
Codename:       jammy
```

If you're on 20.04 or 24.04, stop here and reinstall Ubuntu 22.04. ROS 2 Humble will not work on other versions without headaches.

Section 2 — System prep

```bash
sudo apt update
sudo apt upgrade -y

# Core build / dev tools
sudo apt install -y \
    build-essential cmake git wget curl nano \
    python3-pip python3-argcomplete \
    software-properties-common \
    usbutils i2c-tools \
    net-tools iputils-ping
```

Enable the `universe` repository (required for ROS 2):

```bash
sudo add-apt-repository universe -y
```

Section 3 — Install ROS 2 Humble

3.1 Add the ROS 2 apt repository

```bash
```

```bash
sudo apt install -y curl gnupg lsb-release
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archi
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) ma
    sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
```

3.2 Install ROS 2 desktop + dev tools

```bash
sudo apt install -y ros-humble-desktop ros-dev-tools
```

This is a big install (~2 GB). Go get coffee.

3.3 Install the packages we'll need for this guide

```bash
sudo apt install -y \
    ros-humble-rplidar-ros \
    ros-humble-slam-toolbox \
    ros-humble-nav2-map-server \
    ros-humble-robot-state-publisher \
    ros-humble-joint-state-publisher \
    ros-humble-joint-state-publisher-gui \
    ros-humble-xacro \
    ros-humble-tf2-tools \
    ros-humble-tf2-ros \
    ros-humble-rviz2 \
    ros-humble-laser-filters \
    python3-colcon-common-extensions \
    python3-rosdep \
    python3-vcstool \
    python3-transforms3d
```

3.4 Initialize rosdep

```bash
```

```
sudo rosdep init
rosdep update
```

If `rosdep init` says "already initialized", that's fine — skip it.

3.5 Source ROS 2 automatically

Add to your `~/.bashrc` so every terminal has ROS:

```bash
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "export ROS_DOMAIN_ID=42" >> ~/.bashrc
source ~/.bashrc
```

The `ROS_DOMAIN_ID=42` matches what's in the ARGOS devcontainer and the Pi's planned config — keeping all machines on the same ID prevents weird "I can't see the topics" issues later.

Verify ROS 2 works:

```bash
ros2 --version
ros2 pkg list | grep slam_toolbox
```

Second command should print `slam_toolbox`. If it does, ROS 2 is live.

Section 4 — Get the project files in place

Copy the extracted ARGOS folder to your home directory. The end layout should be:

```
~/argos/
├── Dockerfile
├── README.md
├── docker-compose.yaml
├── docs/
└── ros2_ws/
    ├── argos_control/          (Python source, built by setup.py)
    ├── argos_mission/
    └── src/
        ├── argos_control/       (colcon package)
        ├── argos_mission/
        ├── quadruped_bringup/   (launch files + rplidar config)
        └── quadruped_description/  (URDF)
```

Commands:

```bash
cd ~
# If the zip is in ~/Downloads/Argos.zip:
unzip ~/Downloads/Argos.zip -d ~/
mv ~/Argos/Argos ~/argos           # flatten the double nesting
rm -rf ~/Argos

cd ~/argos
ls ros2_ws/src/                    # should show 4 packages
```

Section 5 — Set up LiDAR hardware access

This is the step people skip and then spend 2 hours debugging.

5.1 Add yourself to the `dialout` group

```bash
sudo usermod -aG dialout $USER
```

You must log out and log back in (or reboot) for this to take effect. Without it, you can see `/dev/ttyUSB0` but can't open it.

5.2 Install the ARGOS udev rule

The project includes a udev rule that creates a stable `/dev/ttyLIDAR` symlink so launch files don't care about USB enumeration order.

```bash
sudo cp ~/argos/ros2_ws/src/quadruped_bringup/config/99-argos-devices.rules \
        /etc/udev/rules.d/

sudo udevadm control --reload-rules
sudo udevadm trigger
```

5.3 Plug in the LiDAR

Plug the RPLiDAR A1M8's USB adapter into your laptop. Wait 2 seconds, then check:

```bash
ls -l /dev/ttyLIDAR /dev/ttyUSB*
```

Expected:

```
lrwxrwxrwx 1 root root 7 Apr 15 14:22 /dev/ttyLIDAR -> ttyUSB0
crw-rw---- 1 root dialout 188, 0 Apr 15 14:22 /dev/ttyUSB0
```

If `/dev/ttyLIDAR` is missing but `/dev/ttyUSB0` exists, the udev rule didn't match. Check the vendor/product ID:

```bash
lsusb | grep -i "cp210\|silicon"
```

Should print something like `Silicon Labs CP210x UART Bridge` with IDs `10c4:ea60`. If it prints different IDs, the lidar is using a different USB bridge chip — edit `/etc/udev/rules.d/99-argos-devices.rules` to match what `lsusb` reports, then rerun `udevadm control --reload-rules && udevadm trigger`.

5.4 Verify you can read the device

```bash
cat /dev/ttyLIDAR
```

Should hang (no output, no error). Ctrl-C to exit. If you get "Permission denied", you either skipped the dialout step or didn't log out/in after it.

Section 6 — Build the workspace

```bash
cd ~/argos/ros2_ws

# Install any workspace-specific deps that rosdep can resolve
rosdep install --from-paths src --ignore-src -r -y

# Build
colcon build --symlink-install
```

Expected tail of the build output:

```
Summary: 4 packages finished [30.2s]
```

6.1 Source the workspace

```bash
echo "source ~/argos/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

6.2 Sanity check

```bash
ros2 pkg list | grep quadruped
```

Should list `quadruped_bringup` and `quadruped_description`.

```bash
ros2 launch quadruped_bringup rplidar.launch.py --show-args
```

Should print the launch arguments (serial_port, frame_id, etc.). If this works, your workspace is built and sourced correctly.

Section 7 — Test: raw LiDAR scans

Terminal 1 — start the lidar:

```bash
ros2 launch quadruped_bringup rplidar.launch.py
```

Expected output:

```
[rplidar_node]: RPLIDAR S/N: ...
[rplidar_node]: Firmware Ver: 1.29
[rplidar_node]: Hardware Rev: 7
[rplidar_node]: RPLidar health status: 0
[rplidar_node]: current scan mode: Standard
```

The motor should spin up and you'll hear a soft whir.

Terminal 2 — verify scans are publishing:

```bash
ros2 topic list | grep scan
ros2 topic hz /scan
```

`ros2 topic hz /scan` should print around `5.5 Hz` (A1M8 scan rate in Standard mode).

```bash
ros2 topic echo /scan --once | head -20
```

You should see `header.frame_id: lidar_link`, an `angle_min`, `angle_max`, and a `ranges` array with 360-ish readings in meters. Values near 0 mean "too close / no return"; values near 8.0 mean "at max range". Block the lidar with your hand — values should drop. If they do, **the lidar works.**

Leave terminal 1 running for the next section.

Section 8 — Test: view scans in RViz

The project includes a prebuilt RViz config.

Terminal 2:

```bash
ros2 launch quadruped_bringup view_lidar.launch.py
```

This launches RViz with `rplidar_view.rviz` already configured. You should see a wireframe 2D scan with points tracing the walls/objects around the lidar.

If RViz opens but no scan appears:

- In RViz, set **Fixed Frame** to `lidar_link` (top left, under Global Options).
- Under LaserScan display, make sure **Topic** is `/scan`.

Kill RViz (Ctrl-C in terminal 2) before proceeding. Leave the lidar running in terminal 1.

Section 9 — Set up the TF tree for SLAM

slam_toolbox needs a valid TF chain: `map → odom → base_footprint → lidar_link`. It provides `map → odom` itself. We need to provide:

- `odom → base_footprint` (we'll use a static identity TF since there's no wheel odometry)
- `base_footprint → base_link → lidar_link` (comes from the URDF via robot_state_publisher)

9.1 Kill whatever's running from section 7

In terminal 1, Ctrl-C the rplidar launch. We're going to restart things in a more structured way.

9.2 Launch everything for SLAM

You'll need 4 terminals. Each terminal must have ROS 2 sourced — if you added the `source` lines to `.bashrc` in earlier steps, this is automatic.

Terminal 1 — LiDAR:

```bash
ros2 launch quadruped_bringup rplidar.launch.py
```

Terminal 2 — Robot description (URDF → TF):

```bash
ros2 launch quadruped_bringup state_publisher.launch.py
```

This parses the URDF and publishes the static transforms for all the robot links. You won't see the legs moving (no joint state publisher active), but `base_link → lidar_link` at `(0.11, 0, 0.055)` gets published.

Terminal 3 — Fake odometry TF:

Since we have no wheel encoders, we publish a static identity TF for odom → base_footprint. The scan matcher in slam_toolbox will figure out real motion from the laser itself.

```bash
ros2 run tf2_ros static_transform_publisher \
    --x 0 --y 0 --z 0 \
    --roll 0 --pitch 0 --yaw 0 \
    --frame-id odom --child-frame-id base_footprint
```

9.3 Verify the TF tree

Terminal 4:

```bash
ros2 run tf2_tools view_frames
```

This writes `frames.pdf` to the current directory. Open it:

```bash
xdg-open frames.pdf
```

You should see a tree: `odom → base_footprint → base_link → {lidar_link, FR_coxa_link, ...}`.

Alternative text-based check:

```bash
ros2 run tf2_ros tf2_echo odom lidar_link
```

Should print a transform updating at ~50 Hz with translation `(0.11, 0.0, 0.055+small)` and identity rotation. If it says "could not find transform", something in the chain is missing — check that terminals 2 and 3 are still running without errors.

Leave all three terminals running for the next section.

Section 10 — Configure SLAM Toolbox

Create a config file tuned for the RPLiDAR A1M8 on a small indoor robot.

```bash
mkdir -p ~/argos/slam_config
nano ~/argos/slam_config/argos_slam_mapping.yaml
```

Paste this in (these parameters are tuned for the A1M8's 8m range, 5.5Hz rate, and a small robot footprint — indoor mapping at 5cm grid resolution):

```yaml
```

```yaml
slam_toolbox:
  ros__parameters:

    # ── Solver ──
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ── Frames / topics ──
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    use_map_saver: true
    mode: mapping

    # ── Runtime ──
    debug_logging: false
    throttle_scans: 1
    transform_publish_period: 0.05
    map_update_interval: 1.0
    resolution: 0.05                          # 5 cm grid
    max_laser_range: 8.0                      # A1M8 reliable max
    minimum_time_interval: 0.2
    transform_timeout: 0.2
    tf_buffer_duration: 30.0
    stack_size_to_use: 40000000
    enable_interactive_mode: true

    # ── Scan matching ──
    use_scan_matching: true
    use_scan_barycenter: true
    minimum_travel_distance: 0.2              # 20 cm before new keyframe
    minimum_travel_heading: 0.2               # ~11.5° before new keyframe
    scan_buffer_size: 10
    scan_buffer_maximum_scan_distance: 10.0
    link_match_minimum_response_fine: 0.1
    link_scan_maximum_distance: 1.5

    # ── Loop closure ──
```

```
    do_loop_closing: true
    loop_search_maximum_distance: 3.0
    loop_match_minimum_chain_size: 10
    loop_match_maximum_variance_coarse: 3.0
    loop_match_minimum_response_coarse: 0.35
    loop_match_minimum_response_fine: 0.45

    # ── Correlation window ──
    correlation_search_space_dimension: 0.5
    correlation_search_space_resolution: 0.01
    correlation_search_space_smear_deviation: 0.1

    # ── Loop search window ──
    loop_search_space_dimension: 8.0
    loop_search_space_resolution: 0.05
    loop_search_space_smear_deviation: 0.03

    # ── Scan matcher ──
    distance_variance_penalty: 0.5
    angle_variance_penalty: 1.0
    fine_search_angle_offset: 0.00349
    coarse_search_angle_offset: 0.349
    coarse_angle_resolution: 0.0349
    minimum_angle_penalty: 0.9
    minimum_distance_penalty: 0.5
    use_response_expansion: true
```

Save (Ctrl+O, Enter) and exit nano (Ctrl+X).

Why these values?

- **resolution: 0.05** — 5 cm/cell is a good balance for an 8m-range lidar in an indoor space. Lower gives finer detail but slower.
- **max_laser_range: 8.0** — matches the A1M8's reliable range; going higher invites noise.
- **minimum_travel_distance: 0.2** — tells SLAM not to add a new keyframe until we've moved 20 cm. Prevents duplicate frames when the robot is still.
- **loop_search_maximum_distance: 3.0** — loop closure looks within 3m of the current estimate for potential reconnections. Tighter than the default because without wheel odometry our drift is bounded by scan matching.

Section 11 — Run SLAM and map the environment

Terminal 4 (the one where you ran tf2_tools — kill it first if still running):

```bash
ros2 launch slam_toolbox online_async_launch.py \
    slam_params_file:=$HOME/argos/slam_config/argos_slam_mapping.yaml \
    use_sim_time:=false
```

Expected startup log:

```
[async_slam_toolbox_node-1] [INFO] Using solver plugin solver_plugins::CeresSolver
[async_slam_toolbox_node-1] [INFO] Node starting up
[async_slam_toolbox_node-1] [INFO] Registering sensor: [Custom Described Lidar]
```

Terminal 5 — RViz for the live map:

```bash
rviz2
```

In RViz:

1. Set **Fixed Frame** to `map` (top left, under Global Options).
2. Click **Add → By topic → /map → Map**. A gray/white/black occupancy grid should appear, centered on the lidar.
3. Click **Add → By topic → /scan → LaserScan**. The live scan appears overlaid.
4. Click **Add → By display type → TF** to see the frames moving.

Actually mapping a room

Since there's no ARGOS frame to drive around yet, the easiest bench test is:

- Put the lidar on a rolling chair or low cart.
- Start at one end of a room.
- Push it **slowly** — about 10–20 cm/second.
- Pause for 2–3 seconds at corners and doorways so the scan matcher gets a clean lock.
- Make a full loop back to your starting point — this triggers loop closure and tightens the map.

You should see walls filling in on the RViz map as you move. The scan matcher will occasionally correct the map when loop closure fires (the map jumps slightly as pose graph optimization runs).

Signs it's working:

- Straight walls stay straight on the map.
- Returning to a previously-mapped area doesn't create a duplicate "ghost" wall (if it does, you moved too fast — slow down).
- `/map` topic updates every ~1 second.

Signs it's failing:

- Map spins or rotates wildly — movement was too fast between scans; restart and go slower.
- Scan shows up in a different location from previous scans — scan matching is getting lost; move back to a known area slowly.

Section 12 — Save the map

Once you've mapped something interesting, save it.

12.1 Save the serialized pose graph (reloadable)

```bash
mkdir -p ~/argos/maps
ros2 service call /slam_toolbox/serialize_map \
    slam_toolbox/srv/SerializePoseGraph \
    "{filename: '$HOME/argos/maps/test_room'}"
```

This creates `test_room.data` and `test_room.posegraph` in `~/argos/maps/`. You can reload these later to continue mapping.

12.2 Save a static occupancy grid (for Nav2)

```bash
cd ~/argos/maps
ros2 run nav2_map_server map_saver_cli -f test_room --ros-args -r map:=/map
```

This produces:

- `test_room.pgm` — the occupancy grid as an image (black=obstacle, white=free, gray=unknown)
- `test_room.yaml` — metadata (resolution, origin, thresholds)

Open the PGM in any image viewer to admire your handiwork:

```bash
xdg-open ~/argos/maps/test_room.pgm
```

Section 13 — Reload a saved map

If you want to continue mapping a previously-saved environment:

```bash
ros2 service call /slam_toolbox/deserialize_map \
    slam_toolbox/srv/DeserializePoseGraph \
    "{filename: '$HOME/argos/maps/test_room', match_type: 2, initial_pose: {x: 0.0,
```

`match_type: 2` means "start in the same pose I saved from". Options are 1 (at saved pose) or 2 (attempt to localize from current scan).

Section 14 — Troubleshooting

The lidar motor spins but `/scan` never publishes

- Check the health message in the rplidar node output. `health status: 0` = good. Anything else means the lidar reports a hardware problem. Unplug/replug USB and try again.
- Try `scan_mode:=Sensitivity` or `scan_mode:=Express` as a launch arg:

```bash
ros2 launch quadruped_bringup rplidar.launch.py scan_mode:=Sensitivity
```

"Permission denied" on /dev/ttyLIDAR

- `groups $USER` should list `dialout`. If it doesn't, redo `sudo usermod -aG dialout $USER` and **log out / log back in** (reboot is the surefire way).

slam_toolbox errors: "Failed to compute odom pose"

- The `odom → base_footprint` TF isn't being published. Check terminal 3 (the static_transform_publisher) is still running.
- Or the frame names in the YAML don't match. The config must use `base_frame: base_footprint` (matches the URDF root) and `odom_frame: odom` (matches the static TF).

Map appears but doesn't update when lidar moves

- Scan matcher can't find a match. Happens when:
  - Motion was too fast (> 0.5 m/s is risky without odom).
  - Environment has long featureless corridors.
  - Returned to a previously-mapped area that's changed.
- Slow movement and pauses fix most cases.

"Could not find transform from 'lidar_link' to 'base_footprint'"

- robot_state_publisher isn't running. Check terminal 2.
- Or it errored on parsing the URDF. Look at terminal 2's stderr.

RViz shows no map / Map display is red

- Click the error icon next to Map in RViz. Usually "No message received" → slam_toolbox isn't publishing `/map`.
- Verify: `ros2 topic echo /map --once | head -20`. Should print header and some cells. If it hangs, slam_toolbox isn't up.

Everything seems to work but map doesn't build

- Check you've actually *moved* at least 20 cm (our `minimum_travel_distance`). A stationary lidar only ever shows one keyframe.
- Check `ros2 topic hz /scan` — if scan rate is <4 Hz, the lidar's USB is having issues. Try a different USB port (avoid USB hubs for first bring-up).

Scan matching drift is accumulating badly

For bench testing without odometry, scan matching drift is inevitable over long runs. Options:

1. Keep runs short (map one room at a time).

2. Always make closed loops — revisit your starting area frequently so loop closure fires.
3. Later, when you have ARGOS moving: pipe joint states and IMU into `robot_localization` to produce `/odometry/filtered`, then set slam_toolbox's odom_frame to read from it.

Section 15 — Quick-start cheat sheet

Once everything is set up, future sessions need only:

```bash
# Terminal 1
ros2 launch quadruped_bringup rplidar.launch.py

# Terminal 2
ros2 launch quadruped_bringup state_publisher.launch.py

# Terminal 3
ros2 run tf2_ros static_transform_publisher \
    --x 0 --y 0 --z 0 --roll 0 --pitch 0 --yaw 0 \
    --frame-id odom --child-frame-id base_footprint

# Terminal 4
ros2 launch slam_toolbox online_async_launch.py \
    slam_params_file:=$HOME/argos/slam_config/argos_slam_mapping.yaml

# Terminal 5
rviz2     # Fixed Frame = map; add /map and /scan displays
```

Save map when done:

```bash
ros2 run nav2_map_server map_saver_cli -f ~/argos/maps/my_map --ros-args -r map:=/ma
```

Section 16 — What's next

Once you're comfortable with this flow:

- **Combine the launches** — create a `slam_bringup.launch.py` in `quadruped_bringup/launch/` that starts lidar + state_publisher + static TF + slam_toolbox

in one command. Pattern it after `sensors.launch.py`.

- **Tune for your environment** — if you're mapping a large open space, increase `loop_search_space_dimension`; if a tight hallway, decrease `correlation_search_space_dimension`.
- **Localization mode** — once you have a good map, launch slam_toolbox with `mode: localization` instead of `mapping` to localize against the saved map without updating it (useful for Nav2 later).
- **Wire in real odometry** — when the ESP32 bridge and IMU are online, use `robot_localization`'s `ekf_node` to fuse IMU + (eventually) leg odometry into `/odometry/filtered`, then remove the static identity TF from terminal 3.

Questions or weird behavior? Screenshot the relevant terminal output and the RViz view, note which section you were on, and ping Chase.